ВИЗУАЛНО ПРОГРАМИРАНЕ С JAVA

Упражнение 6

Контактите идват от база данни: драйвер, една връзка за цялото приложение, клас за заявките, ResultSet.

ПЪЛЕН ФАЙЛ

Това е пълният вариант: теория, всички задачи, домашни, справки. **За самото занятие** има кратък лист „Упражнение 6 — за часа“ — задачите 1 – 5 в него са същите.

КАКВО ВИ ТРЯБВА ДНЕС

Проектът `vpj-gXX` и хранилището `vpj` от упражнение 5 — днес продължаваме в тях, нов проект не създаваме.

Класът `Contact` с конструктор с пет параметъра, гетъри, сетъри, `toString()` и `getProperties()`. Формата `LoginForm` с `main()` и вход `admin` / `123456`. Формата `ContactForm` със списъка `contacts`, `lblStatus`, `addSamples()` и `miList` „Списък с контакти…“.

Формата `ContactListForm(List<Contact>)` с `tblContacts` и модела ѝ, `lstContacts`, `lblCount`, `lblDetails`, радио бутоните в `bgCity`, `chkEmail`, `btnDelete`, `popRow`; методите `show(...)`, `selected()`, `deleteSelected()`, `selectedCity()`, `applyFilter()`.

Два файла от e-learn: `chinook.db` и `Connect.java`. Файлът `chinook.db` — само този от e-learn: в интернет има и друга версия на същата база, с която днешният код не работи.

JDBC драйвер — нова Maven зависимост.

Част	Време	Какво правим
—	5 мин	Подготовка: git pull, стартиране, вход, двата файла от e-learn
1	10 мин	Теория: 1.3 пътят до файла и празната база, 1.4 една връзка от main() до формата, 1.5 ResultSet. Базата в DB Browser — показно. Останалите точки са справка по време на задачите.
2	75 мин	Задачи 1 – 5 (6 – 7 ако остане време), с commit след всяка

1. Теоретична част

1.1. Какво добавяме днес

Досега контактите се създаваха в `addSamples()` и губехме промени при затваряне на програмата. От днес ще идват от **база данни** — файла `chinook.db`, в който има 59 клиенти. Приложението се променя на четири места:

- **JDBC драйвер** — библиотеката, през която Java говори с базата; dependency в `pom.xml`;
- **една връзка** — отваря се в `main()` и ще я предаваме през конструкторите на формите;
- **клас за обръщение към базата** — `ContactRepository`, единственото място с SQL;
- **резултат от заявката** (`ResultSet`) — отговорът на заявката, който ще превръщаме в обекти `Contact`.

Таблицата, списъкът, филтърът и изтриването **не се променят**. Сменя се само откъдето идват данните: един ред на мястото на `addSamples()`. Базата засега само се чете.

1.2. SQLite и базата Chinook

SQLite е база данни в един файл. Няма сървър, който трябва да е пуснат: библиотеката се вгражда в самото приложение и чете и пише във файла директно. Затова SQLite се използва най-често за мобилни приложения, но понякога и за настолни програми.

`chinook.db` е примерна база на магазин за музика. Таблица `Customers` има 13 колони; от тях ще ползваме 6:

Колона	Тип	В Contact
`CustomerId`	число	`id` — новото поле от задача 4
`FirstName`, `LastName`	текст	`firstName`, `lastName`
`City`, `Country`	текст	`city`, `country`
`Email`	текст	`email`

Петте текстови колони са точно петте полета на `Contact`. `CustomerId` е **първичният ключ** на таблицата — номер, който е различен за всеки ред и по който един клиент се намира еднозначно, дори да има двама с едно и също име.

Съдържанието на базата се разглежда с **DB Browser for SQLite**: Open Database → `chinook.db`. Разделът **Database Structure** показва таблиците и колоните им, **Browse Data** — редовете на избрана таблица, а в **Execute SQL** се пишат заявки.

1.3. Драйвер и пътят до файла

JDBC позволява на Java приложение да работи с база данни, благодарение на общ набор от класове (`Connection`, `PreparedStatement`, `ResultSet`, `SQLException`) в пакета `java.sql`. За всеки тип база има **драйвер**, който изпълнява тези класове за нея. За SQLite това е `org.xerial:sqlite-jdbc` — блок в `pom.xml`, както FlatLaf.

Връзка с база се отваря, посредством **connection string** — текст от вида:

```
jdbc:sqlite:chinook.db       // технология : вид база : файл
```

Пътят до файла е **относителен** — търси се в папката, от която е стартирана програмата. При стартиране от NetBeans това е главната папка на проекта, там, където е `pom.xml`. Пълен път от вида `D:/Desktop/chinook.db` работи само на вашия компютър.

> **ЛИПСВАЩИЯТ ФАЙЛ НЕ Е ГРЕШКА ЗА SQLITE**
>
> Ако `chinook.db` не е там, където програмата го търси, SQLite **създава нов празен файл** със същото име и връзката се отваря успешно. Грешка няма. Тя идва чак при първата заявка: `no such table: Customers` — в празната база няма таблици.
>
> Затова `main()` проверява дали файлът съществува **преди** да отвори връзката, и ако го няма — показва пълния път, по който е търсен. Ако някога видите в проекта `chinook.db` с размер 0 байта, това е такъв празен файл: изтрийте го.

1.4. Една връзка за цялото приложение

Ще използваме клас `Connect`. В конструктора си отваря връзката към `chinook.db`, `getConnection()` я връща, `close()` я затваря. Ако връзката не може да се отвори, конструкторът хвърля `SQLException`.

Връзката се отваря **веднъж**, в `main()`, преди първата форма. После се подава на формите през конструктора — както списъкът с контакти се подава на `ContactListForm` от упражнение 4:

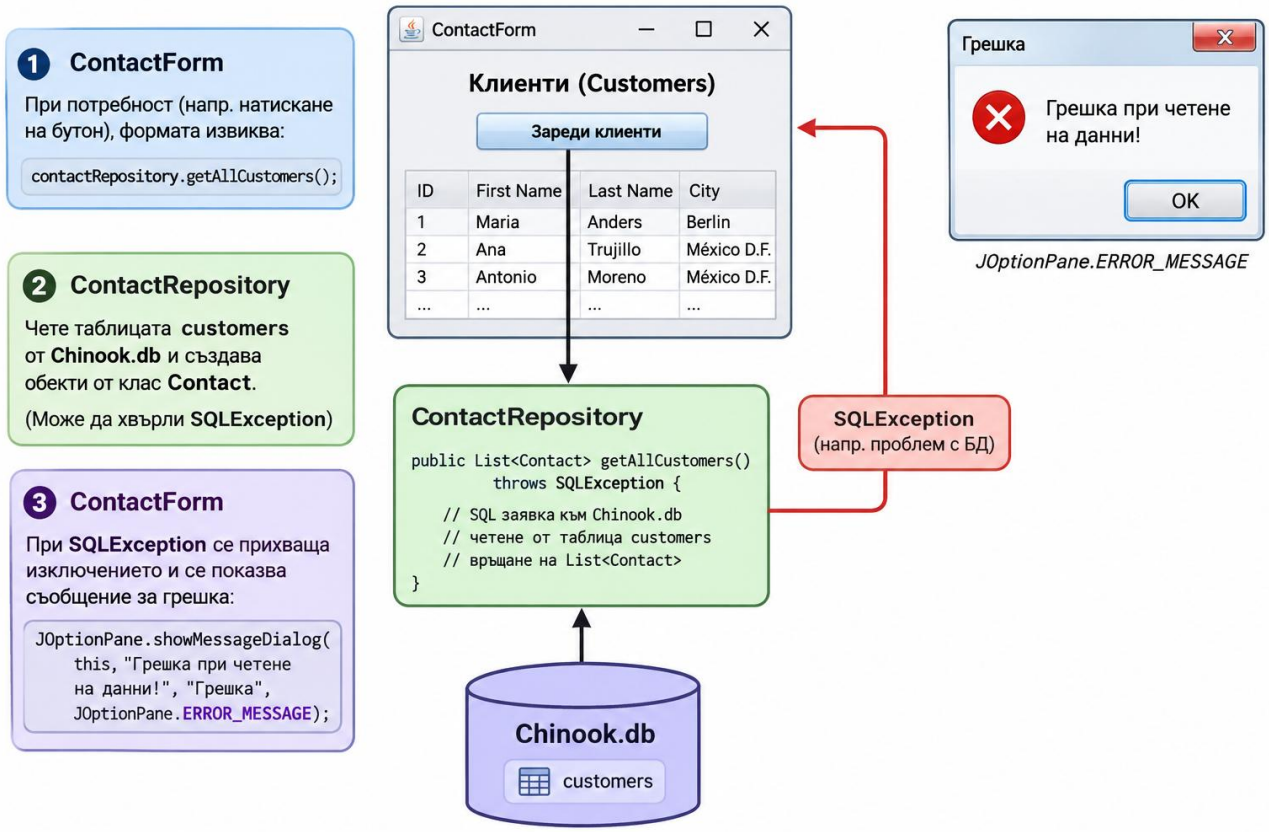
```
LoginForm.main()  → new Connect() → new LoginForm(db)
   btnLogin        → new ContactForm(db)
ContactForm(db)   → contactRepo = new ContactRepository(db)
```

Формите не отварят и не затварят връзка. Затварянето е закачено в `main()`:

```
Runtime.getRuntime().addShutdownHook(new Thread(() -> db.close()));
```

Shutdown hook е код, който Java изпълнява при спиране на програмата — независимо коя форма е затворена последна и как. `() -> db.close()` е **ламбда** — код, подаден като стойност: Java го пази и го изпълнява, когато програмата спира.

> **ФОРМАТА НЕ ПИШЕ SQL**
>
> Всички заявки към таблица `Customers` са в един клас — `ContactRepository`. Формата вика неговите методи (`count()`, `loadAll()`) и получава число или списък с обекти. Класът се казва по модела (`Contact`), а в заявките стои името на таблицата (`Customers`).
>
> Repository класът получава `Connect` в конструктора и пази `Connection` от него. Методите му хвърлят `SQLException` нагоре — прихваща ги формата, защото само тя може да покаже съобщение на потребителя.



Поток на данните в приложението.

1.5. Заявка и ResultSet

Заявка се изпълнява в три стъпки: текстът на заявката става `PreparedStatement`, той се изпълнява с `executeQuery()`, а отговорът е `ResultSet`:

```
try (PreparedStatement stmt = conn.prepareStatement(sql);
     ResultSet rs = stmt.executeQuery()) {
    while (rs.next()) {                        // следващ ред, докато има
        String name = rs.getString("FirstName");
    }
}                                              // stmt и rs се затварят тук
```

`ResultSet` е отговора на заявката под форма на таблица — редове, всеки с колоните от `SELECT`. Но той е **курсор**: в началото стои **преди** първия ред; `rs.next()` го премества на следващия и връща `true`, докато има ред. Движи се само напред — обходен веднъж, не може да се обходи пак. Затова редовете се копират в списък от обекти, с който формата после прави каквото ѝ трябва.

Стойност от текущия ред се взема по **името на колоната**: `rs.getString("City")`, `rs.getInt("CustomerId")`. Или по номер, като колоните се броят **от 1**: `rs.getInt(1)` — удобно за `SELECT COUNT(*)`, където колоната няма име.

try с ресурси. Нещата в скобите след `try` се затварят автоматично в края на блока — и когато всичко мине, и когато някой ред хвърли изключение. `catch` липсва нарочно: методът е обявен с `throws SQLException` и изключението стига до формата.

Текстът на заявка от няколко реда се пише между три двойни кавички `"""` — многоредов низ, в който новите редове са част от текста.

1.6. Контактът получава ID

Ред от базата се познава по `CustomerId`, затова `Contact` получава поле `id`. Контактите, създадени от формата — с „Добави“ или с диалога — не са в базата и нямат такъв номер; за тях `id` е 0.

Конструкторът с пет параметъра остава — ползват го `btnAdd` и `newContact()`. Добавя се втори, с шест, който **вика първия** и после записва `id`:

```
this(firstName, lastName, city, country, email);   // другият конструктор на класа
```

`this(...)` като извикване е друг конструктор на същия клас. Пишете го на първия ред на конструктора — така петте полета се попълват на едно място, а не на две.

Справка

Клас / метод	Какво прави
`new Connect()`	отваря връзка към `chinook.db`; хвърля `SQLException`, ако не може
`db.getConnection()`	връща `Connection` — от него се правят заявките
`conn.prepareStatement(sql)`	заявка, подготвена от текст
`stmt.executeQuery()`	изпълнява `SELECT`, връща `ResultSet`
`rs.next()`	курсорът на следващия ред; `false`, ако няма
`rs.getString("Колона")`, `rs.getInt("Колона")`	стойност от текущия ред по име на колона
`rs.getInt(1)`	стойност по номер на колона, от 1
`Files.exists(Path.of("chinook.db"))`	има ли такъв файл спрямо папката на стартиране
`path.toAbsolutePath()`	пълният път — за съобщението, когато файлът липсва

2. Практическа част

2.1. Подготовка (5 мин)

1. Ако сте работили на друга машина: `git pull`.
2. Стартирайте с F6, влезте с `admin` / `123456`, отворете „Списък с контакти…“ — седемте контакта трябва да са в таблицата и в списъка.
3. Свалете от e-learn `chinook.db` и `Connect.java` — за задача 1.

2.2. Задачи

Правилото от предишните занятия важи и днес: **една задача — един commit** с описателно съобщение. Задачи 1 – 5 са за час; 6 – 7 са ако остане време.

> **СКЕЛЕ И ЯДРО**
>
> **Скелето** — готов блок код за копиране с //TODO редове за дописване. **Ядрото** — подсказка за всяко TODO, пишат се и се обясняват в час.
>
> Местата във всяка форма са три: **над конструктора** (полета), **в конструктора** и **над реда `// Variables declaration`** (нови методи). В нов клас — **над последната затваряща скоба**.

Задача 1. Драйвер, база и Connect (8 мин)

1. Отворете `pom.xml` (Project Files → pom.xml). В `<dependencies>`, под блока на FlatLaf, поставете:

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.53.4.0</version>
</dependency>
```

2. Clean and Build (Shift + F11). Maven сваля драйвера; в Output конзолата трябва да излезе `BUILD SUCCESS`.
3. Сложете `chinook.db` в **главната папка на проекта** — където е `pom.xml`, не в `src` и не в `resources`. Проверете в Window → Files: файлът е на едно ниво с `pom.xml`.
4. `Connect.java` — в пакета `app`, при другите класове. **Без промени.**
5. Commit: `git add .`, `git commit -m "задача 1: драйвер, база, Connect"`. Базата влиза в хранилището веднъж — днес само се чете и файлът не се променя.

Проектът се компилира, а при стартиране нищо не се е променило. Връзката още не се отваря никъде.

Задача 2. Връзката от main() до формите (12 мин)

Връзката се отваря в `main()` и се подава на `LoginForm`, от нея на `ContactForm`. Започнете от `LoginForm`. Докато не стигнете до `ContactForm`, NetBeans ще подчертава с червено — компилаторът показва точно местата, откъдето се стига до формата.

Скеле — в `LoginForm.java`: импортите, полето над конструктора, новият вид на конструктора и първият ред на конструктора (останалото в него си стои), а **старият `main()` се изтрива** и на негово място идва този:

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.sql.SQLException;
import javax.swing.JOptionPane;

// над конструктора
private final Connect db;

// конструкторът – нов подпис и първи ред
public LoginForm(Connect db) {
    this.db = db;
    initComponents();
    // … каквото вече има
}

public static void main(String[] args) {
    FlatLightLaf.setup();
    java.awt.EventQueue.invokeLater(() -> {
        Path file = Path.of("chinook.db");
        if (!Files.exists(file)) {
            JOptionPane.showMessageDialog(null,
                    "Няма файл с база:\n" + file.toAbsolutePath(),
                    "Грешка при стартиране", JOptionPane.ERROR_MESSAGE);
            return;
        }
        Connect db;
        try {
            db = new Connect();
        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(null,
                    "Няма връзка с базата.\n" + ex.getMessage(),
                    "Грешка при стартиране", JOptionPane.ERROR_MESSAGE);
            return;
        }
        Runtime.getRuntime().addShutdownHook(new Thread(() -> db.close()));
        // TODO 1: отворете формата за вход с връзката
    });
}
```

В `ContactForm.java` — **същите импорти и същият `main()`**: копирайте го от `LoginForm` и сменете само TODO (вижте ядрото). Конструкторът получава подпис `public ContactForm(Connect db)`; тялото му засега не се пипа.

Ядро — трите места, на които се създава форма, трябва да станат:

```
// TODO 1 – в main() на LoginForm
new LoginForm(db).setVisible(true);

// TODO 2 – в main() на ContactForm
new ContactForm(db).setVisible(true);

// в btnLoginActionPerformed – при успешен вход, вместо new ContactForm()
new ContactForm(db).setVisible(true);
```

Проверете: F6, вход — всичко е както преди. После в Window → Files преименувайте `chinook.db` на `chinook1.db` и стартирайте: диалог с пълния път, по който е търсена базата. Върнете името на `chinook.db`.

Задача 3. ContactRepository и първа select заявка (10 мин)

Нов клас: десен бутон върху пакета `app` → New → Java Class → **ContactRepository**. В него — поле за връзка `conn` от клас `Connection`, конструктора да приема отворена връзка от `Connect` и да я предаде на `conn` чрез `db.getConnection()` и един метод `count()`, изхвърлящ SQLException, който изпраща select заявка за броя на клиентите в таблица Customers.

Скеле — целият клас, на мястото на генерирания от NetBeans:

```
package app;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class ContactRepository {

    private final Connection conn;

    public ContactRepository(Connect db) {
        this.conn = db.getConnection();
    }

    public int count() throws SQLException {
        String sql = "SELECT COUNT(*) FROM Customers";
        try (PreparedStatement stmt = conn.prepareStatement(sql);
             ResultSet rs = stmt.executeQuery()) {
            // TODO 1: курсорът на единствения ред, после числото от колона 1
        }
    }
}
```

В `ContactForm.java` добавете поле `contactRepo` от клас `ContactRepository`, на първия ред на конструктора на формата създайте contactRepo, като подадете връзката към базата в конструктора му и създайте метод `loadFromDb`, който изпълнява заявката за брой клиенти и изписва резултата в `lblStatus`:

```
// над конструктора
private final ContactRepository contactRepo;

// първи ред на конструктора, преди initComponents();
contactRepo = new ContactRepository(db);

// последен ред на конструктора
loadFromDb();

private void loadFromDb() {
    try {
        // TODO 2: в lblStatus – колко контакта има в базата
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(this, ex.getMessage(),
                "Грешка в базата", JOptionPane.ERROR_MESSAGE);
    }
}
```

Ядро:

```
// TODO 1
rs.next();
return rs.getInt(1);

// TODO 2
lblStatus.setText("В базата има " + contactRepo.count() + " контакта.");
```

Проверете: след входа `lblStatus` казва „В базата има 59 контакта.“ Това е проверката, че връзката, файлът и таблицата са наред — едновременно.

> **АКО ИЗЛЕЗЕ „no such table: Customers“**
>
> Файлът е намерен, но в него няма такава таблица. Или е празен файл с размер 0 байта (т. 1.3), или е друга версия на Chinook, в която таблиците са в единствено число. Вземете `chinook.db` от e-learn.

Задача 4. Контактът получава ID (5 мин)

В `Contact.java` добавете ново private поле за id, после създайте нов конструктор, приемащ всички 6 данни, като в него извикате конструктора, приемащ 5-те, и добавите присвояване на ID. После добавете getter за id. Има ли нужда от setter?

```
// в новия конструктор с 6 параметъра
this(firstName, lastName, city, country, email); //извиква конструктора с 5 параметъра
this.id = id;
```

Проектът трябва да се компилира и нищо видимо да не се е променило: `btnAdd` и диалогът „Нов контакт“ ползват стария конструктор и остават непроменени.

Задача 5. Контактите идват от базата (15 мин)

Добавете втори метод в `ContactRepository` — всички клиенти като списък от `Contact`, сортирани по фамилно име. В ContactForm той ще замени `addSamples()`.

В `ContactRepository`, под `count()`, създайте метод `loadAll()`, който изхвърля `SQLException` и връща `List<Contact>`:

```
public List<Contact> loadAll() throws SQLException {
    String sql = """
            //query
            """;
    //създайте динамичен списък от Contact, който ще пълните с резултата. Отворете нова
връзка с try с ресурси и накрая извън блока на try return на динамичния списък
}
```

Данните се четат в while цикъл, като върти по rs.next(), връща false, когато резултатите свършат:

```
while (rs.next()) {
    list.add(new Contact(
            rs.getInt("CustomerId"),
            rs.getString("FirstName"),
            rs.getString("LastName"),
            rs.getString("City"),
            rs.getString("Country"),
            rs.getString("Email")));
}
```

После в ContactForm в `loadFromDb()` извикайте `contacts.addAll(contactRepo.loadAll());` и изтрийте `addSamples();` от конструктора (самият метод може да остане)

```
contacts.addAll(contactRepo.loadAll());
```

Стартирайте и отворете списъка: 59 контакта в таблицата и в списъка вдясно, подредени по фамилия — Almeida е първи. Това е `ORDER BY` от заявката; таблицата не е сортирана. Таблицата, списъкът, изтриването и контекстното меню работят, без да променена `ContactListForm`.

Щракнете „София“ — трябва всички да изчезнат, защото в базата няма клиенти от София, Пловдив и Варна. Чекбоксът „Само с имейл“ трябва да работи за всички — всичките 59 имат имейл.

> **ДО ТУК САМО ЧЕТЕМ ДАННИ ОТ БАЗАТА**
>
> Ако изтриете контакт, затворите програмата и я стартирате пак — контактът ще се появи отново. `deleteSelected()` трие обекта от `contacts`, който е копие на базата в паметта на приложението, направено при стартиране. Аналогично „Добави“ и „Нов контакт“ добавят данни само докато приложението не бъде затворено.

> **ЗАДАЧИ 6 – 7: АКО ОСТАНЕ ВРЕМЕ**
>
> Задачи 1 – 5 са основата: драйвер, връзка от `main()`, repository клас, `ResultSet` → списък от обекти. 6 и 7 допълват.

Задача 6. Колона „Код“ (6 мин, ако остане време)

В `ContactListForm` към `COLUMNS` добавете **в края** `"Код"`, а в `getProperties()` на `Contact` — **в края** `id`. В края, защото на други места колоните се търсят по номер — например градът е колона 2 в `editCity()`.

Стартирайте: кодовете са от 1 до 59, но не по ред — таблицата е подредена по фамилия. Добавете контакт с „Нов контакт…“: кодът му е 0.

Задача 7. Какво помни базата (4 мин, ако остане време)

Отворете `chinook.db` в DB Browser, Browse Data → `Customers`, и намерете клиента с `CustomerId` 12 — Roberto Almeida. В програмата изтрийте същия контакт, затворете я и в DB Browser натиснете F5 (опресняване). Редът е там. Сега отговорете в едно изречение, като коментар над `deleteSelected()`: **от кое изтрива този метод?**

2.3. Преди да си тръгнете

`git push` и отворете страницата на хранилището в браузър. Трябва да виждате `chinook.db`, `Connect.java`, новия `ContactRepository.java`, променените `pom.xml`, `Contact.java`, `LoginForm.java` и `ContactForm.java` и поне пет нови commit-а с разбираеми съобщения.

> **КЪДЕ СМЕ**
>
> Връзката се отваря веднъж в `main()` и минава през конструкторите. Заявките са в `ContactRepository`; формата вика `count()` и `loadAll()`. Контактите идват от базата, а таблицата, списъкът и филтърът не са пипнати. Базата засега само се чете.

3. Домашна работа

Правило: **една задача — един commit**. Изпратете всичко с `git push` преди следващото занятие. Първо — задачи 6 – 7, ако не са направени в час.

8. Заявки в DB Browser (таб Execute SQL, изпълнение с Ctrl + Enter). Напишете ги в текстов файл `sql/homework6.sql` в проекта и го комитнете. Отговорите за самопроверка са в скоби.

- всички клиенти от Бразилия — `WHERE Country = 'Brazil'` (5 реда);
- държавите, без повторения, по азбучен ред — `SELECT DISTINCT Country … ORDER BY Country` (24 реда);
- брой клиенти по държави, най-многото отгоре — `COUNT(*)` и `GROUP BY Country` (първа е USA с 13).

9. Сменете `ORDER BY` в `loadAll()` на `Country, LastName`. Какво се промени в таблицата и в списъка? Кое не се промени, и защо `ContactListForm` не трябваше да се променя?

10. Радио бутоните с градове от базата. В Navigator преименувайте `rbSofia`, `rbPlovdiv`, `rbVarna` на **rbParis**, **rbBerlin**, **rbLondon**, сменете текстовете им и низовете в `selectedCity()` на `Paris`, `Berlin`, `London`. Всеки град дава по два контакта.

Задача 11. Повишена трудност: курсорът не се връща

В `loadAll()`, след цикъла `while`, добавете пак същия цикъл, който брои редовете в променлива и я отпечатва с `System.out.println`. Колко отпечата? После на мястото на втория цикъл извикайте `rs.beforeFirst();` — което на пръв поглед трябва да върне курсора в началото. Какво казва изключението и какво означава думата FORWARD в него?

Махнете добавеното. Отговорете в коментар над `loadAll()`: защо методът връща `List<Contact>`, а не самия `ResultSet`? (Подсказка: какво става с `rs`, когато свърши `try` блокът?)

Приложение 1. Кодът от днес на един екран

```
// --- pom.xml ------------------------------------------------------------
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.53.4.0</version>
</dependency>

// --- main(): една връзка за цялото приложение ---------------------------
Path file = Path.of("chinook.db");
if (!Files.exists(file)) { /* диалог с file.toAbsolutePath() */ return; }
Connect db;
try { db = new Connect(); }
catch (SQLException ex) { /* диалог с ex.getMessage() */ return; }
Runtime.getRuntime().addShutdownHook(new Thread(() -> db.close()));
new LoginForm(db).setVisible(true);

// --- веригата -----------------------------------------------------------
public LoginForm(Connect db)   { this.db = db; initComponents(); … }
new ContactForm(db).setVisible(true);              // при успешен вход
public ContactForm(Connect db) {
    contactRepo = new ContactRepository(db);
    initComponents();
    // …
    loadFromDb();                                  // на мястото на addSamples()
}

// --- ContactRepository --------------------------------------------------
public ContactRepository(Connect db) { this.conn = db.getConnection(); }

public int count() throws SQLException {
    String sql = "SELECT COUNT(*) FROM Customers";
    try (PreparedStatement stmt = conn.prepareStatement(sql);
         ResultSet rs = stmt.executeQuery()) {
        rs.next();
        return rs.getInt(1);
    }
}

public List<Contact> loadAll() throws SQLException {
    // SELECT CustomerId, FirstName, LastName, City, Country, Email
    // FROM Customers ORDER BY LastName, FirstName
    while (rs.next()) {
        list.add(new Contact(rs.getInt("CustomerId"),
                rs.getString("FirstName"), /* … */ rs.getString("Email")));
    }
}

// --- ContactForm --------------------------------------------------------
private void loadFromDb() {
    try {
        lblStatus.setText("В базата има " + contactRepo.count() + " контакта.");
        contacts.addAll(contactRepo.loadAll());
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(this, ex.getMessage(), "Грешка в базата",
                JOptionPane.ERROR_MESSAGE);
    }
}

// --- Contact ------------------------------------------------------------
public Contact(int id, String firstName, String lastName,
               String city, String country, String email) {
    this(firstName, lastName, city, country, email);
    this.id = id;
}
```

Преписвайте кода, вместо да го копирате от файла — виж бележката в упражнение 1, т. 1.1.

Приложение 2. Странно поведение и какво значи то

Какво	Причина
Диалог „Няма файл с база“ с някакъв път	`chinook.db` не е в показаната папка. Сложете го до `pom.xml` (Window → Files)
`No suitable driver found for jdbc:sqlite:chinook.db`	драйверът липсва: блокът не е в `pom.xml` или проектът не е компилиран след добавянето му — Clean and Build
`BUILD FAILURE` с „Could not resolve dependencies“	няма интернет при първата компилация, или в блока има грешка в `groupId` / `version`
`no such table: Customers`	празен файл с размер 0 байта (т. 1.3) или друга версия на Chinook — вземете `chinook.db` от e-learn
В проекта се появи `chinook.db` с размер 0 байта	програмата е стартирана без базата и SQLite е създал празна. Изтрийте файла
`no such column: 'Email'`	името в `getString(...)` не е колона от `SELECT` — липсва в заявката или е написано различно
`constructor ContactForm in class ContactForm cannot be applied to given types`	някъде още стои `new ContactForm()` без `db` — в `main()` или в `btnLoginActionPerformed`
`missing return statement` в `count()`	`return` е извън `try` или липсва — трябва да е след `rs.getInt(1)`
Списъкът показва 66 контакта	`addSamples()` още се вика — 59 от базата плюс седемте примерни
Броят расте при всяко отваряне на списъка	`loadFromDb()` се вика от `miList`, а не от конструктора — `addAll` добавя наново
„София“, „Пловдив“, „Варна“ дават празна таблица	в базата няма клиенти от тези градове — домашна 10
„Само с имейл“ не променя нищо	всичките 59 клиенти имат имейл
Изтритият контакт е пак там след рестарт	`deleteSelected()` маха от `contacts` — копието в паметта, не базата
Добавеният контакт изчезва след рестарт	същото: `contacts.add` не пише в базата
Таблицата е подредена по фамилия, без да е щракнато заглавие	`ORDER BY LastName, FirstName` в заявката
Кодът на нов контакт е 0	създаден е с конструктора с пет параметъра — не е от базата
`ResultSet is TYPE_FORWARD_ONLY`	`ResultSet` е курсор само напред — задача 12
Грешката от базата излиза чак след входа	нормално: заявките са в конструктора на `ContactForm`, който се създава при успешен вход